VIVA PREPARATION: CAMPUS LOST & FOUND

Likely questions and short answers. Practise saying them in your own words.

1. What problem does your project solve?

There is no searchable place on campus to report lost and found items, and no way for a finder to check that a claimant really owns the item. The app gives both: searchable posts and a proof question before hand-over.

2. Why a proof question instead of just "I lost it"?

Anyone can say an item is theirs. The finder sets a question only the owner could answer, for example the sticker on a bottle, and sees the answer before approving.

3. How are passwords stored?

As salted scrypt hashes using Node's crypto module. A random 16-byte salt is stored with the hash, and logins are checked with a timing-safe comparison.

4. How does login stay valid between requests?

On login the server creates a random token and stores it in the sessions table. The browser sends it in the Authorization header; logout deletes it.

5. How do you prevent SQL injection?

Every query uses prepared statements with ? placeholders, so user input is never concatenated into SQL.

6. How do you prevent XSS?

The front end escapes every piece of user text (&, <, >, quotes) before putting it into the page.

7. Can one person claim the same item many times?

No. The claims table has a UNIQUE (item_id, claimant_id) constraint, and the API returns 409 for a second attempt. You also cannot claim your own post.

8. Who can read the messages?

Only the poster and the claimant of that claim. The API checks this on every read and write and returns 403 to anyone else.

9. Why SQLite and not MySQL?

For a single-campus app SQLite is enough, needs no separate server, and is built into Node 24. The SQL is standard, so moving to MySQL or PostgreSQL later means changing

SAMPLE

Sample document by Final Commit for a fictional college. Names in square brackets are placeholders.

the driver, not the design.

10. How are photos handled?

The browser resizes them to at most 800 pixels and compresses them to JPEG before upload, so a 4 MB phone photo becomes about 60 to 120 KB.

11. What does the demo on GitHub Pages use instead of the server?

An in-browser store built on IndexedDB with the same functions as the HTTP store, so the same front end runs with or without the server.

12. How did you test it?

An automated script starts the server with an empty database and checks 18 behaviours, including permission rules. The front end was tested by walking the full flow in a browser.

13. What happens when an admin hides a post?

Its hidden flag is set. Students no longer see it in lists or by link; the campus office still can, and can unhide it.

14. What are the limitations?

No email or SMS notifications, photos are stored inside the database, and there is no integration with the college login system.

15. What would you add next?

Notifications when a matching item is posted, image storage in a separate bucket, college single sign-on, and simple image matching between lost and found posts.

SAMPLE

Sample document by Final Commit for a fictional college. Names in square brackets are placeholders.